

Topic 8: Multiple Inheritance and Ambiguity

8.1 Multiple Inheritance

- A class can inherit from more than one base class
- Syntax: `class C : public A, public B { }`;
- C inherits all public and protected members from both A and B
- Useful when a class genuinely IS-A combination of two types (e.g., a FlyingCar is both a Car and an Aircraft)

```
class Flyable {
public:
    void fly() { cout << "Flying" << endl; }
};

class Swimmable {
public:
    void swim() { cout << "Swimming" << endl; }
};

class Duck : public Flyable, public Swimmable {
public:
    void quack() { cout << "Quack!" << endl; }
};
```

8.2 Ambiguity in Multiple Inheritance

- Problem: if both base classes define a function with the same name, the compiler cannot decide which to call
- Solution: use the scope resolution operator to specify: `obj.A::show()`;

```
class A { public: void show() { cout << "A" << endl; } };
class B { public: void show() { cout << "B" << endl; } };
class C : public A, public B { };

C c;
// c.show();           // ERROR: ambiguous
c.A::show();           // OK
c.B::show();           // OK
```

8.3 The Diamond Problem

- Occurs when two base classes share a common grandparent: $A \leftarrow B, A \leftarrow C, B+C \leftarrow D$
- D would inherit two copies of A's members -- one from B's path and one from C's path
- Solution: virtual inheritance -- `class B : virtual public A { }`;
- Virtual base class ensures only one shared copy of A's members exists in D

```
class A { public: int x = 0; };
class B : virtual public A { };
class C : virtual public A { };
class D : public B, public C { };

D d;
d.x = 5; // OK -- only one copy of x
```

8.4 When to Use Multiple Inheritance

- Prefer single inheritance + interfaces (abstract classes) over heavy multiple inheritance
 - Multiple inheritance is suitable when mixing in independent capabilities (logging, serialisation)
 - Keep multiple-inherited hierarchies shallow to avoid complexity
-